表格结构识别与单元格对齐

摘要：本文讨论从学术文档中恢复表格结构的工程做法。与正文抽取不同，表格的难点不在识字，而在判断哪些字符属于同一行、同一列，以及在被跨页与合并单元格打断之后如何恢复出可用的二维结构。

1　引言。很多系统把表格当作普通文本处理，结果是行列关系在抽取阶段就丢失了，下游无论怎样分块与检索都无法恢复。本文的主张是：表格必须作为结构化对象处理，宁可整体保留也不要把它的内部结构抹平。

2　行列边界的判定。表格的行边界通常由文本的纵向间距给出，而列边界需要统计整块区域的空白分布。我们采用的规则是：水平投影的空白带宽度超过阈值才被认定为列边界。单一字符宽的间隙不能作为边界依据，因为相邻列之间的默认留白常常与之接近，容易切出错误的碎列。

3　阈值的选择。空白带阈值与字号强相关：字号越大，同一语义间隙对应的像素宽度越大。因此在多字号混排的文档里，固定像素阈值必然在某一档上出错。可行的做法是按行内字号的中位数归一化之后再比较，代价是需要先完成字号估计。

4　跨页表格。学术文档中长表格经常跨页，第二页起通常只有列内容而没有表头。我们的处理原则是：跨页表格必须在表头重复之后才允许合并，否则应当把两页的表格视为两个独立对象，并在溯源信息里记录它们原本属于同一张表。

5　单元格内换行。表格单元格宽度有限，长文本会在格内折行，而折行在抽取后表现为独立的文本行。如果直接把每一行当作一个单元格，列数就会膨胀、对齐随之错乱。正确做法是先把折行还原。具体地，单元格内的换行要先还原成同一格再判断列归属，这一步应当在列切分之前完成。

6　合并单元格。表头里常见的跨列标题会让首行的列数多于数据行，而纵向合并会让某些行的起始列缺失。合并单元格会导致后续行的列数少于表头列数。遇到这种情况应当显式记录合并跨度，而不是用空字符串补齐，因为空字符串与真正的空单元格无法区分。

表　1　常见缺陷与处理方式。折行被当作新行　先还原再切列　；合并单元格缺列　记录跨度　；跨页无表头　拆成两张表　；列边界误判　按字号归一化　。

7　数字与单位。表格里的数值常带有单位、正负号与百分号，分词器容易把它们切成多个记号。在表格场景下这类错误影响更大，因为一个被拆开的数值会跨到相邻列去。因此表格单元格在进入分块器之前必须先做数字与单位的紧耦合还原。

8　表注与脚标。表注通常位于表格下方，与表格之间只有一行间距，很容易被误并入最后一行。识别依据是表注往往以固定前缀开头，且其字体小于表格正文字体。把表注并入表格会污染数据行，把数据行误判为表注则会丢失最后一行，两种错误都要避免。

9　表格与上下文的关联。表格的意义高度依赖它前后的正文段落，孤立的一张表往往无法解释。因此分块时应当把表格与其前一段说明绑定在同一个检索单元里，而不是让表格单独成块——单独成块的表格在被召回时缺少解释性语境。

10　评测口径。表格抽取的质量不能只看字符正确率，因为结构错误远比字符错误严重。我们使用三个指标：行数是否正确、列数是否正确、以及单元格内容的精确匹配率。其中行数错误会导致整表语义改变，应当赋予最高权重。

11　与分块器的配合。表格进入分块器时应当携带一个结构标记，告诉分块器这一段不可按句切分。否则分块器会按标点把表格行切成若干片段，而表格行本身往往缺少标点，结果是被当作一个超长句整体处理，反而错过了与正文合并的机会。

12　召回与精确的取舍。表格识别的召回比精确更重要因为漏行会改变整表语义。这一取舍与正文抽取相反：正文漏一句通常只损失局部信息，而表格漏一行会让整张表的行列对应关系整体错位。因此我们允许表格识别出现少量多余行，再由后处理按列数一致性过滤。

13　与拍平处理的对比。把表格拍平成一行的做法会让不同列的内容被缝在同一句里，产生语义上

错误但读起来通顺的片段，这种错误最难被发现，因为它不会引发任何异常，只会在回答里悄悄出现。相比之下，保留结构最多是检索时匹配不上，属于可观测的失败。

14　实现要点。我们的实现分为四步：先做区域切分得到候选表格块，再做行聚类，然后按空白带切列，最后做单元格归属与合并跨度恢复。每一步都产出中间结构，便于在出错时定位到具体阶段，而不是只能看到最终的错误结果。

15　结论。表格抽取的核心是保住结构：宁可整体成块、宁可保留多余的边界，也不要把二维结构压成一维文本。结构一旦丢失就无法在下游恢复，而多余的边界信息总可以在后续步骤里被过滤掉。

16　中英混排的列宽问题。当中文与英文出现在同一列时，英文字符的平均宽度明显小于中文，按字符数估算的列宽会系统性偏大，导致列边界落在英文列的内部。我们的处理是按视觉宽度而不是字符个数统计空白带，并把中英字符按经验权重折算后再比较。

17　多行表头。很多论文的表头本身就有两行：第一行是大类，第二行是细分列。把它当作两行数据会导致列名与数据错位，因此需要识别出连续的表头行并做纵向合并，合并后的列名用短横线连接，作为该列的最终名称参与检索。

18　边框与无线表。学术文档里三线表很常见，横向分隔线只出现在表头与表尾；也有完全没有边框的表。前者可以用线条辅助定位行边界，后者只能依赖间距统计。实现上应当把线条信息当作可选信号而不是必需条件，否则无线表会整体失败。

19　关于视觉模型的取舍。用视觉语言模型直接读表在某些场景下效果更好，但它引入了不可复现的因素：同一张表在不同调用里可能得到不同的结构。评测集需要可复现，因此我们把视觉路线排除在评测口径之外，只保留它作为线上增强的可选项。

20　表格块的表示形式。我们给表格块加上结构前缀，标明它是表格、包含多少行多少列，随后才是内容本身。这样检索时表格与正文可以被区分开，回答阶段也能显式说明证据来自表格而非段落，便于用户核对。

21　数值列的对齐。数值通常右对齐、文本左对齐，这个排版习惯本身就是一种结构信号。当空白带统计出现歧义时，对齐方式可以作为打破平局的依据；但反过来不能依赖它定位列边界，因为并非所有表格都遵守这一习惯。

22　单位与量纲。同一列里混用不同单位会让数值失去可比性。我们不做单位换算，只在单元格里保留原始写法，并在表头里保留单位说明，把解释权交给读者与下游模型，避免因为自动换算引入无法追溯的数值错误。

23　表内引用。表格单元格里常常出现引用标记，它们不是内容的一部分。保留它们会污染列名与数值，删除它们又可能破坏溯源，我们的折中是保留字符但在检索匹配时忽略，并在结构信息里单独记录该单元格含引用。

24　与页码的配合。表格的溯源要求比正文更高，因为它必须同时给出行列位置。因此表格块记录的是页码加表序号加行列坐标，仅凭页码无法定位到具体单元格，这在核对引用时会造成很大困扰。

25　性能占比。在实际文档上，表格识别的时间开销远小于文本抽取与编码，通常不到整篇处理时间的一成。因此优化重点不应放在表格识别的速度上，而应放在正确率上，尤其是行数与列数这两个足以改变语义的指标。

26　回归测试。表格抽取的退化往往是静默的：结构错了不会抛异常，只会让下游回答出错。因此我们为每种缺陷形态各准备一个小样本，断言其行列数与关键单元格内容，把这类退化变成可在提交前拦住的失败。

28　表格块的表示。表格块的表示形式会直接影响它能否被检索到。把整张表压成一个长字符串时，查询里的单个术语会在极长的文本里被平均数效应淹没；按行拆成多个小块又会让列名与数值分离。可行折中是按行组切块并让每块带上表头。

29　表头重复的成本。给每个行组都带上完整表头会增加冗余，但这份冗余换来的是每个块都能自解释。在检索场景里，块的自解释性比存储效率重要得多，因为用户看到的是单个块而不是整张表。

３０　溯源粒度。表格的溯源必须给出行列坐标而不只是页码。行列表现在溯源的唯一性上尤其明显：同一页上可能有多张表，仅凭页码无法区分，因此表格块必须记录表序号与行列范围，才能让用户核对到具体单元格。

３１　与正文的引用关系。正文里说「如表三所示」时，表格与正文之间就建立了一条引用关系。把这条关系保留下来，检索到正文段落时就能一并给出对应表格，这是纯文本表示无法提供的能力。

３２　质量回归。表格抽取的缺陷大多不抛异常，因此回归样本必须覆盖跨页、合并单元格、无线表与表内换行四类形态，并对每类断言行列数与若干关键单元格，而不是只断言字符数量。

３４　单元格的类型推断。把「3」解释成数字还是字符串会影响下游的排序与统计。在检索场景下这一区分并不重要，但在生成阶段模型会把字符串当作原文照抄、把数字当作可比较的量，因此类型信息应当以最小必要的方式保留。

３５　表内空值。空单元格与缺失列是两件事：前者表示该项无数据，后者表示结构被切错。二者在文本里都表现为空白，因此必须依赖列数一致性来区分，而不能靠单元格内容判断。

３６　表格与页码的联合定位。表格的溯源必须给出行列坐标而不只是页码。当同一页存在多张表时，仅凭页码定位会指向整页，用户无法确认引用的是哪一张，因此表序号和行列范围是必需信息而不是可选信息。

３７　结论补充。把表格当作结构化对象，意味着它需要自己的质量指标、自己的回归样本、以及自己在检索里的表示形式。把这三件事做齐之后，表格才真正可被检索与引用，而不只是“恰好被抽出来的那一段文本”。